Agentic Coding Course

Syllabus & Lab Catalog — D2L Brightspace (Self-Paced)

Format: Self-paced online · 3 phases × 6 weeks (18-week suggested pace) · one weekly Zoom for support **Modules:** 7 (00–06) · **Learning objects / labs:** 70 · **Inference:** shared on-prem vLLM endpoint (2× RTX A6000, single shared key)
Document type: Approval / overview edition — full syllabus plus a condensed lab summary per module.

Part A — Syllabus

A.1 Delivery model

The course is delivered **self-paced** in D2L Brightspace. Three pillars:

1. **Self-paced content.** Students progress on their own; advancement is gated by completing labs, not by attendance.
2. **A lab for every learning object.** Each lesson is one learning object and ships with a paired hands-on **lab** that produces a verifiable artifact (screenshot, command output, repo/commit link, or auto-checked quiz).
3. **Weekly Zoom (support, not a gate).** One live session per week reviews that week's objectives, answers questions, and walks through labs. Students ahead of or behind the suggested pace still advance.

The 18-week / 3-phase schedule below is a **suggested pace** that orients students and sets each weekly Zoom's focus — it does not lock content.

Lab pattern (applied to every learning object): *Objective → lab task → submit artifact → completion unlocks the next object.* Entry `setup-check` lessons become D2L **Checklists**; each module's `final-checkpoint` becomes a **phase-gate** that must be passed to advance.

Machine prerequisite (required to enroll): a laptop running **Windows 10/11 with WSL2 (Ubuntu)** or **macOS**. The course lives in a Linux/Unix shell throughout — Windows students get that through WSL (set up in Module 00), and Mac students use the built-in terminal. Chromebooks and tablets are **not** supported. All coursework runs against the college's **internal** server (VS Code + Continue.dev point at the shared vLLM endpoint on the internal network); Windows students may alternatively work entirely over SSH into that server (Module 00, lesson 06).

A.2 Suggested 18-week pace

Phase 1 — Foundations (weeks 1–6) · Modules 00, 01, 02 · 27 labs

Week	Module	Lessons / Labs	Focus
1	00	00–04	IDE concept, VS Code install/tour, integrated terminal, extensions
2	00 → 01	00:05–07, 01:00–01	WSL/SSH remote, Module 00 phase-gate; terminal setup + navigation
3	01	02–06	Making things, files, sudo/apt, installing software, permissions
4	01	07–11	Networking, processes, pipes/grep, users/groups, phase-gate
5	02	00–03	Docker setup-check, install, hello-world, nginx
6	02	04–06	Images vs containers, VS Code Docker sidebar, phase-gate + buffer

Phase 2 — Building real apps (weeks 7–12) · Modules 03, 04 · 25 labs

Week	Module	Lessons / Labs	Focus
7	03	00–04	Prereqs, Python venv, install FastAPI, hello world, path/query params
8	03	05–08	Pydantic bodies, routers, dependency injection, containerizing
9	03	09–12	Compose + Postgres, SQLAlchemy, error handling, phase-gate
10	04	00–04	Prereqs, git config, init/first commit, status/add/commit, diff/log
11	04	05–09	Branches, undoing, Gitea + SSH, pushing, cloning/pulling
12	04	10–11	Pull requests, phase-gate + buffer

Phase 3 — AI-assisted development (weeks 13–18) · Modules 05, 06 · 18 labs

Week	Module	Lessons / Labs	Focus
13	05	00–03	Meet the shared vLLM endpoint, point Continue.dev at it, verify chat
14	05	04–06	Configure Continue, chat with `@` -context, autocomplete
15	05	07–09	Inline edit, agent mode, phase-gate
16	06	00–03	Prereqs, install Node (nvm), init HeroUI app, first component
17	06	04–07	CORS + fetch, create-item form, extend with Continue, phase-gate
18	—	capstone	Combine FastAPI + HeroUI + Continue against the shared model; demo at the final Zoom

> **Module 05 note:** The shared vLLM endpoint replaces the former per-student local GPU + Ollama setup. Lessons 05/00–04 are reframed around connecting Continue.dev to the shared endpoint (endpoint URL + shared key, no per-student token); lessons 05/05–09 (chat, autocomplete, inline edit, agent mode, checkpoint) are unchanged in substance.

A.3 D2L Brightspace structure

Course concept	D2L construct
Phase (×3)	**Unit** (top-level Content container)
Week (×6 per phase)	**Module/Lesson** sub-container (suggested-pace grouping)
Learning object (lesson)	**Topic** (HTML content) **+ paired Lab**
Lab submission	**Assignment (Dropbox)** for an artifact, or **Quiz** for an auto-check
`setup-check` lesson	**Checklist** (module entry gate)
`final-checkpoint` lesson	**Phase-gate Quiz/Assignment** (required to advance)
Self-paced progression	**Release Conditions** + **completion tracking**
Weekly Zoom	**Calendar event** + **Announcement**; recording posted as a Topic
Shared endpoint URL + key	Released D2L content item (posted to the cohort)

A.4 Shared inference infrastructure (owned)

The college runs **one shared inference server** that every student connects to from Continue.dev (and, later, their own API code). It replaces the per-student local-GPU + Ollama setup — the hardware is **owned and already in place**, not a procurement decision. Target concurrency: **~16–30 students**.

Hardware: a single server with **2× NVIDIA RTX A6000 (48 GB each, ~96 GB aggregate VRAM)**.

Serving: vLLM exposes an **OpenAI-compatible API** with continuous batching — a good fit for the bursty, spread-out load of self-paced students. The large model runs **tensor-parallel across both A6000s** (`--tensor-parallel-size 2`).

Access: students point Continue.dev at the shared endpoint over the **campus network / Tailscale** (e.g. `http://gpu.college.local:8000/v1`) using a **single shared API key** — no per-student tokens and no LiteLLM proxy to operate.

Models served — US-origin, no China-origin models (Qwen/DeepSeek/etc. excluded per policy; Llama satisfies it):

Use in Continue	Model	Origin / License	Status
Autocomplete (only)	**Llama 3.1 8B Instruct**	Meta (US) / Llama Community	Live now
Chat / inline edit / agent	**Llama 3.3 70B Instruct** (quantized, tensor-parallel across both A6000s)	Meta (US) / Llama Community	Coming online

---

Part B — Lab Catalog

Each learning object below is paired with a lab. *Setup-check* labs gate module entry; *final-checkpoint* labs are phase-gates that must be passed to advance.

Phase 1 — Foundations

Module 00 — VS Code & IDE Fundamentals

#	Lesson	Objective	Lab task
00	What is an IDE?	Understand editor vs terminal vs IDE; install VS Code	Install VS Code and open it; submit a screenshot of the welcome screen
01	VS Code UI Tour	Learn the parts of the VS Code window	Open a folder, use the command palette, set font size to 16
02	The Integrated Terminal	Locate and use the terminal pane	Open the terminal and run `echo` commands successfully
03	Extensions Overview	Understand how extensions add features	Install the Python extension by Microsoft
04	The Docker Extension	Install the Docker extension for Module 02	Install the Docker extension; sidebar populates after Module 02
05	Connecting to WSL	Install Remote — WSL; connect (Windows students)	Connect to WSL and run `uname -a` showing Linux
06	Connecting over SSH	Install Remote — SSH; connect to a remote machine	Connect to the host; run `hostname` and `whoami`
07	Final Checkpoint	Prove mastery of the IDE + remote connections	Submit a screenshot of a remote terminal with `whoami` and `pwd`

Module 01 — Ubuntu Terminal Fundamentals

#	Lesson	Objective	Lab task
00	Setup Check	Confirm the Ubuntu environment is ready	Run version, network, and sudo checks; confirm all pass
01	Where Am I?	Navigate the filesystem with `pwd`, `cd`, `ls`	Move between directories; use `cd -` and `cd ~`; list root dirs
02	Making Things	Create the working directory tree	Create `~/docker-course/` with notes, practice, projects folders
03	Looking Inside Files	Read and edit files with `cat`, `head`, `tail`, `nano`	Create a notes file, append lines, edit with nano, verify 9 lines
04	Sudo and System Updates	Update the system with `sudo`	Run `sudo apt update` and `sudo apt upgrade -y` successfully
05	Installing Software	Install prerequisite packages	Install curl, wget, git, tree, etc.; verify each version
06	Permissions Drills	Read, set, and change permissions and ownership	Create a 755 executable script; change ownership with `chown`
07	Networking Drills	Confirm the network is ready for Docker pulls	Ping 8.8.8.8, reach hub.docker.com, check ports with `ss`
08	Processes and Services	Inspect processes and service status	Run `ps` / `top` / `htop`; confirm SSH and cron services active
09	Pipes and Grep	Chain commands with pipes; search with `grep`	Use pipes to count items; grep files and logs for patterns
10	User and Group Setup	Understand users and groups	Check `id` / `groups`; preview `usermod -aG docker`
11	Final Checkpoint	Prove full terminal proficiency	Run 33 commands from memory across all topics; all checks pass

Module 02 — Docker Fundamentals

#	Lesson	Objective	Lab task
00	Setup Check	Confirm readiness for Docker install	Verify Linux, sudo, prerequisites, Module 01 complete
01	Install Docker Engine	Install Docker; join the docker group	Run Docker without sudo; `docker info` works
02	Your First Container	Understand image vs running container	Run hello-world; list images and containers with `docker ps`
03	Running a Real Image	Run nginx and reach it from the browser	Run nginx on port 8080; load the page and view logs
04	Images vs Containers	Distinguish images from containers	Run three nginx containers from one image
05	The VS Code Docker Sidebar	Manage containers/images from VS Code	Run, stop, remove a container via the UI; view logs
06	Final Checkpoint	Prove Docker proficiency	Run nginx, curl it, screenshot the sidebar, clean up

Phase 2 — Building real apps

Module 03 — FastAPI Development

#	Lesson	Objective	Lab task
00	Prerequisites Check	Verify Python, pip, Docker, curl	Run all version checks and log to notes
01	Python Virtual Environment	Create and activate a project venv	Set up `.venv`, activate it, add to `.gitignore`
02	Installing FastAPI	Install FastAPI and Uvicorn	Add fastapi/uvicorn to `requirements.txt`
03	Hello World	Run a FastAPI server with hot-reload	Create `main.py` with `/` route; curl and browse `/docs`
04	Path and Query Params	Add typed, validated parameters	Build `/items/{item_id}`; test type validation (422)
05	Request Bodies with Pydantic	Accept validated JSON bodies	Create `POST /items` with an `Item` model; POST via curl/Swagger
06	Multiple Routes and Routers	Organize routes with `APIRouter`	Refactor items/users into `routers/` with prefixes
07	Dependencies and Injection	Share logic with `Depends()`	Wire `get_current_user` with X-Token header auth
08	Containerizing FastAPI	Package the app into an image	Write a Dockerfile, build, run with port mapping
09	Docker Compose and Postgres	Orchestrate FastAPI + Postgres	Write `docker-compose.yml`; verify hot-reload via bind mount
10	Database with SQLAlchemy	Persist data via the ORM	Add models + `get_db()`; query and insert rows
11	Error Handling and Status Codes	Return correct HTTP codes	Add status codes; raise `HTTPException`; test with curl
12	Final Checkpoint	Build a resource end-to-end	Implement `/profiles` CRUD with auth + persistence

Module 04 — Git & Gitea

#	Lesson	Objective	Lab task
00	Prerequisites Check	Confirm Git + the FastAPI project	`git --version`; verify the project files exist
01	Configure Git	Set identity and defaults	Run `git config --global`; verify with `--list`
02	Init and First Commit	Turn the project into a repo	`git init`, set `.gitignore`, add, first commit
03	Status, Add, Commit Cycle	Master the everyday workflow	Make 3+ commits with different staging patterns
04	Diff and Log	Read history and diffs	`git diff`, `git log --oneline`, compare commits
05	Branches	Create and merge branches	Branch, commit, merge (fast-forward and true merge)
06	Undoing Things	Discard, unstage, amend, reset	Drill `restore`, `restore --staged`, `commit --amend`, `reset`
07	Gitea Account and SSH	Authenticate to Gitea via SSH	Create an SSH key, add it to Gitea, verify with `ssh -T`
08	Pushing to Gitea	Push and sync a remote	Create the repo, `git push -u origin main`, round-trip
09	Cloning and Pulling	Clone and resolve conflicts	Clone to a second dir; cause and resolve a conflict
10	Pull Requests	Open, review, and merge a PR	Push a feature branch; open a PR; comment; merge
11	Final Checkpoint	Run the full PR cycle from memory	Add an `/info` route via branch → PR → review → merge

Phase 3 — AI-assisted development

Module 05 — Continue.dev on the Shared vLLM Endpoint

#	Lesson	Objective	Lab task
00	Prerequisites Check	Confirm Modules 00–04 still work; VS Code + Continue ready	Run version/compose/FastAPI checks; log passes
01	The Inference Endpoint	Understand the shared server (2× A6000, vLLM) and the models it serves	Note the endpoint URL, the 8B (autocomplete) and 70B (chat/agent) models, and their roles
02	Connect to the Endpoint	Verify the shared endpoint responds	`curl` the `/v1/models` endpoint with the shared key; confirm the model list
03	Explore the Models	Identify the chat and autocomplete models served	Send a test chat completion to the 70B via curl; note 8B is autocomplete-only
04	Install + Configure Continue.dev	Point Continue.dev at the shared endpoint	Install the extension; write `config.yaml` (openai provider, endpoint URL, both Llama models); chat works
05	Chat	Bring context in with `@file` / `@codebase` / `@diff` / `@terminal`	Drill the four `@` providers against the FastAPI repo
06	Autocomplete	Write a real route with tab autocomplete	Write a `PUT /items/{id}` route using ghost-text suggestions
07	Inline Edit	Request code changes with `Ctrl+I`	Rename a function, add validation, extract a helper, write a test
08	Agent Mode	Make multi-file changes with tool-call approvals	Add a `/status` route + `tags` field via agent; reject bad steps
09	Final Checkpoint	Build a feature end-to-end with Continue	Implement `/items/search?q=` ; ship via a Gitea PR

Module 06 — HeroUI Frontend

#	Lesson	Objective	Lab task
00	Prerequisites Check	Confirm FastAPI, Continue, Gitea, clean Node	Test services; check `which node` is clean
01	Install Node.js with nvm	Install nvm + LTS Node	Run the nvm installer; verify `node` / `npm` / `npx`
02	Init the HeroUI App	Scaffold a Next.js + HeroUI app	`npx heroui-cli@latest init`; start the dev server on :3000
03	Tour + First Component	Build a first HeroUI component	Replace the page with Button + Card + Input + reset
04	CORS + Fetch Items	Fetch FastAPI data; fix CORS	Add CORS middleware; render items as Cards
05	Create-Item Form	POST a new item and refresh inline	Build a name/price/isOffer form; submit creates an item
06	Extend with Continue.dev	Add a feature via agent mode	Add a Delete button + confirmation modal; review diffs
07	Final Checkpoint	Ship a feature end-to-end	Add a live client-side search filter; commit and merge

Open decisions (for the college / instructor)

- Pick the weekly **Zoom day/time** and start date.
- **Lab grading:** completion-only vs. graded (affects the D2L gradebook).
- **Endpoint distribution:** how the shared endpoint URL + key reach students (released D2L item).
- **Network access:** campus-network-only vs. Tailscale for off-campus students.
- **70B rollout:** confirm the 70B is loaded and stable before Phase 3 opens; interim fallback if not.
- **Confirm the model-origin policy** (China-origin exclusion vs. a named allowlist) — Llama satisfies the current reading.
- Choose **manual D2L build** vs. **IMS Common Cartridge** import for cohort 1.